ROS2 Report

Sean D' Almeida
Driverless Subsection
Formula Manipal
Manipal, Udupi
seanalmeidaofficial@gmail.com

ABSTRACT This report investigates the primary computation graph primitives—Nodes, Topics, Services, Actions, and Dynamic Parameters—alongside low-level execution concepts including Executors, Threading Models, Component Composition (zero-copy intra-process transport), and Lifecycle (Managed) State Machines. To illustrate these theoretical mechanics, a fully executable, production-style Python (rclpy) publisher node is implemented and analyzed line-by-line. Finally, the report examines the diagnostic toolkit (rviz2, rqt_graph, ros2bag) essential for system verification. This work provides both a foundation for entry-level robotics engineers and an advanced architectural reference for industrial autonomous software design.

Keywords: *ROS2, Nodes, Topics, Actions, Services, Colcon, Quality of Service*

I. INTRODUCTION

The Robot Operating System 2 (ROS 2) is an open-source, industrial-grade robotics middleware framework. Despite its name, ROS 2 is not an operating system in the traditional sense (such as Linux or Windows); rather, it operates as a middleware layer above host operating systems (Linux, macOS, Windows, and Real-Time Operating Systems). It provides hardware abstraction, device drivers, intra-process communication mechanisms, package management, and debugging utilities designed to build complex, scalable robotic system.

ROS2 was developed by the Open Robotics as a successor to ROS. While ROS 1 revolutionized academic robotics, it suffered from structural architectural limitations that prevented its deployment in commercial, real-time, and safety-critical environments. Key differences were:

- *Central Discovery*: While ROS1 required roscore, Every node must register with roscore to find other nodes. If roscore crashes, the entire system stops working, making it a single point of failure. Uses a peer-to-peer model. Nodes discover each other automatically using DDS dynamic discovery mechanisms. There is no central master, meaning if one node fails, the rest of the network continues to communicate normally.
- *Transport Protocol*: Built on custom, application-layer transport protocols called TCPROS and UDPROS. These were custom wrappers built on top of standard TCP/UDP, specifically designed for ROS, which limited interoperability with other non-ROS systems. ROS2Uses the globally recognized, industrial-standard DDS (Data Distribution Service) protocol. DDS handles data serialization, transport, and routing out of the box, allowing ROS 2 to leverage highly optimized, commercial-grade communication software.
- *OS Support*: ROS1 was Linux-centric whereas ROS2 has native multiplatform support
- *Real-time capability*: ROS 2 provides native support for hard real-time execution, meaning it can guarantee that critical tasks (like braking a robotic car before a collision) happen at the exact millisecond required, whereas ROS 1 can only make a "best effort" without external modifications.
- *Quality of Service*: ROS 2 introduces granular, QoS profiles, allowing developers to customize exactly how data is handled for different scenarios, whereas ROS 1 strictly locks you into a rigid choice between reliable TCP (reliable however prone to data backlogs) or best-effort UDP (fast however data loss prone).

II. CORE ARCHITECTURE AND COMPUTATIONAL GRAPH

The Computation Graph represents the entire live network of a robotic system, acting as a blueprint of all active processes and how data flows between them in real time. It is a true peer-to-peer (P2P) network. Once 'nodes' discover each other via DDS, they establish direct, point-to-point connections through topics, services or action topologies.

A. Nodes

A Node is a fundamental computational unit in the Robot Operating System 2 (ROS 2) framework. It is designed to perform a specific logical task within a robotic system, promoting modularity, maintainability, and reusability. Each node typically encapsulates a particular functionality, such as acquiring sensor measurements, processing camera images, estimating the robot's position, controlling actuators, or planning a trajectory. By dividing a complex robotic application into smaller, independent components, ROS 2 enables the system to be developed, tested, and maintained more efficiently.

In ROS 2, nodes are implemented using client libraries such as rclpy for Python and rclcpp for C++. In Python, a node is commonly created by defining a class that inherits from rclpy.node.Node, whereas in C++, it is typically implemented using the rclcpp::Node class or its derived classes. These classes provide the functionality required to interact with the

ROS 2 communication infrastructure, including creating publishers and subscribers, defining services and actions, and managing timers and callbacks

B. Topics (Publish-Subscribe Model)

Topics are one of the primary communication mechanisms in the ROS2, enabling asynchronous data exchange between nodes through a publish-subscribe communication model. They are primarily used to transmit continuous streams of data, such as sensor measurements, robot velocity commands, and system status information.

This communication mechanism allows multiple nodes to exchange information without requiring direct knowledge of one another, thereby promoting modularity, flexibility, and scalability in robotic applications. In ROS 2, a topic is identified by a unique name that represents the type or purpose of the information being transmitted and each topic carries messages of a specific data type, ensuring that publishers and subscribers exchange information in a defined and consistent format.

A publisher is a component within a node that sends messages to a particular topic. It generates data whenever new information becomes available or according to a predefined publishing frequency

A subscriber, on the other hand, is a component within a node that registers interest in a topic to receive its messages. When a new message is received, the subscriber invokes a callback function to process the data

One of the main characteristics of the publish-subscribe model is its asynchronous and decoupled communication structures: Publishers do not need to know the identities of the subscribers receiving their messages, and subscribers do not need to know which specific publishers are providing the data.

A single publisher can transmit messages to multiple subscribers, while multiple publishers may publish to the same topic, provided that their messages use compatible types and communication settings. Similarly, a single node can publish to several topics and subscribe to multiple topics simultaneously. This many-to-many communication capability makes topics suitable for complex robotic systems in which multiple components need to share information concurrently. Topic communication is generally non-blocking from the application's perspective. A publisher can publish a message without waiting for a subscriber to process it, while a subscriber processes incoming messages through callback functions managed by the ROS 2 execution system. Consequently, publishers and subscribers can operate at different rates and do not need to execute in synchronisation. However, this does not guarantee that every published message will be received or processed; actual delivery behaviour depends on factors such as the Quality of Service (QoS) settings, network conditions, message buffering, and executor scheduling.

C. Services (Request-Response Model)

Services are another fundamental communication mechanisms ROS 2, enabling nodes to communicate through a request-response model. Unlike topics, which are primarily designed for continuous data streaming, services are intended for operations in which a client sends a request to another node and receives a corresponding response after the requested operation has been processed. This mechanism is particularly suitable for tasks that require a specific result or confirmation.

A ROS 2 service consists of two main components: a Service Server and a Service Client. The service server provides a specific operation and waits for incoming requests, while the service client initiates communication by sending a request to the server. Each service is associated with a defined interface that specifies the structure of both the request and the response, ensuring that the communicating nodes use a consistent data format. When a request is received, the server executes the corresponding callback function, performs the required operation, and returns a response to the client.

Service communication can be handled synchronously or asynchronously from the client's perspective. In a synchronous interaction, the client waits for the service response before proceeding with subsequent operations. In an asynchronous interaction, the client sends the request and continues executing other tasks while waiting for the response, which can be handled later through a future or callback mechanism. ROS 2 supports asynchronous service calls through its client APIs, which are useful for preventing unnecessary blocking in time-sensitive robotic applications. Although services follow a request-response model, the underlying communication is managed by the ROS 2 middleware.

Services are generally intended for short-duration operations with a clearly defined result. They are appropriate when a node needs to perform a discrete operation and return a response within a reasonable period. However, services do not inherently guarantee deterministic execution time.

Another important characteristic of services is their one-to-one interaction pattern: each request is directed to a particular service endpoint, and the client receives a response corresponding to that request. This differs from topics, where a publisher can distribute messages to multiple subscribers without requiring individual responses. By providing a structured request-response mechanism, services enable nodes to invoke specific functionality in other nodes while maintaining software modularity and separation of responsibilities.

D. Actions(Client-Server Model)

Designed to manage long-running tasks that require continuous progress updates, asynchronous execution, and the ability to cancel an operation before it finishes. Unlike topics, which primarily facilitate continuous data streaming, and

services, which are generally intended for short-duration request-response operations, actions are suitable for tasks that may take a significant amount of time to complete and whose execution progress needs to be monitored. Common applications include autonomous navigation, robotic arm movement, trajectory execution, and other complex robotic operations.

In action involves two main components: an Action Client and an Action Server. The action client initiates a task by sending a goal that specifies the desired operation, while the action server receives the goal, executes the requested task, and communicates its progress and outcome to the client. ROS 2 actions use a structured communication interface consisting of three principal elements: a goal, feedback, and a result. This structure allows the client to monitor the execution of a task and respond to changing requirements while the operation is in progress.

Goal: The goal represents the task specification submitted by the action client to the action server. It contains the information required to define the desired operation. For example, in a mobile robot navigation system, the goal may specify the target position and orientation that the robot must reach. Upon receiving a goal, the action server determines whether it can accept the request. If accepted, the server begins executing the task and tracks its progress until completion, cancellation, or another terminal condition occurs

Feedback: Feedback consists of periodic updates sent by the action server to the client while the goal is being executed. These updates provide information about the current state or progress of the task, allowing the client or other system components to monitor execution. For instance, a navigation action server may report the robot's current position, the distance remaining to the target, or other progress-related measurements. Feedback enables the system to observe task execution without having to wait for the final result. The frequency and content of feedback depend on the action's implementation.

Result: The result is the final response returned by the action server when the goal reaches a terminal state. It contains information about the outcome of the requested task, such as whether the robot successfully reached its destination or whether the operation failed. If the goal is canceled, the server completes the cancellation process and reports the corresponding terminal status and result according to the action interface. The result allows the client to determine the final outcome and decide whether further action is necessary.

One of the most important characteristics of ROS 2 actions is their support for cancellation and preemption. A client can request cancellation of an ongoing goal if the task is no longer required or if the system needs to respond to a change in operating conditions.

Actions operate asynchronously, allowing the client to continue performing other computations while the server executes the requested task. This is particularly important in robotic systems, where several processes, such as sensor monitoring, localization, obstacle detection, and motion planning, must operate concurrently. By providing feedback and allowing cancellation, actions enable the system to monitor long-running operations and adapt to changing circumstances without blocking the entire application

E. Parameters

Parameters are configurable values associated with individual nodes in ROS 2 (Robot Operating System 2) that allow users and developers to adjust a node's behaviour without modifying its underlying source code. These parameters are typically stored and managed locally within each node and are used to define important operational settings, such as maximum motor velocity limits, controller gain constants, sensor calibration values, communication settings, and control-loop frequencies. By allowing these values to be configured separately from the program logic, parameters make robotic systems more flexible, adaptable, and easier to maintain.

One of the key features of parameters in ROS 2 is the ability to modify them dynamically during runtime. This enables developers and operators to fine-tune a robot's behaviour while the system is running, without necessarily recompiling the software or restarting the node. For example, the proportional, integral, and derivative (PID) controller gains can be adjusted to improve the stability and responsiveness of a robotic system, while velocity and acceleration limits can be modified to suit different operating conditions. Such adjustments can be performed through command-line tools, graphical interfaces, or programmatically through ROS 2 interfaces.

ROS 2 also provides mechanisms for declaring, retrieving, updating, and validating parameters, allowing nodes to manage configuration values in a structured manner. Parameters may be assigned default values in the code or loaded from external configuration files, commonly written in YAML format. This makes it easier to maintain consistent configurations across different runs and deployments. Additionally, parameter validation and callback mechanisms can be implemented to ensure that changes satisfy predefined constraints before they are applied.

F. Difference between Topics and Services

The primary difference between topics and services lies in their communication patterns and intended purposes. Topics are designed for continuous, asynchronous data exchange, making them suitable for real-time sensor monitoring and telemetry. Services, on the other hand, are intended for request-response interactions where a client requests a particular operation and receives a corresponding result.

Topics generally allow publishers to transmit messages without waiting for subscriber responses, whereas service

clients typically expect a response to each request. Additionally, topics use .msg files to define message structures, while services use .srv files to define both request and response structures.

In conclusion, topics and services complement each other within ROS 2. Topics support the continuous exchange of information between distributed nodes, while services provide a structured mechanism for requesting specific operations and obtaining results. Selecting the appropriate communication mechanism depends on whether an application requires continuous data transmission or a discrete request-response interaction.

III. COLCON

In ROS 1, the build process relied primarily on the Catkin build system, which was closely integrated with CMake. Although this worked well for many ROS packages, it presented some limitations when managing projects that used different programming languages, build systems, or operating environments.

The main reasons for developing Colcon were:

- Support for Multiple Build Systems.
- Reduced Dependence on ROS
- Improved Cross-Platform Support

Colcon was developed as a flexible and extensible build tool that coordinates different build systems through a unified workspace workflow. Depending on the available extensions and package configuration, it can support build systems such as CMake and Python packaging tools.

A. Core Architecture and Build Flow

1. Package Discovery: Colcon scans the workspace, usually the src/ directory, to identify available packages and determine which build tools are required.
2. Dependency Analysis: Colcon examines the declared dependencies between packages to determine the correct build order. If one package depends on another, the required package must generally be built first.
3. Build Execution: Colcon invokes the appropriate build tools for each package.

IV. WORKSPACE STRUCTURE

A typical ROS 2 workspace contains four main directories: src/, build/, install/, and log/. Each directory serves a specific purpose in organizing source code, managing the build process, storing the resulting software, and recording diagnostic information.

- src/ contains the source code and package files created or maintained by the developer. These include program files, package manifests, configuration files, and other resources required for development. This directory is primarily managed by the user, who can add, modify, or remove packages as needed.
- build/ stores intermediate build files, compiler outputs, generated files, and build-system configuration files created during compilation. These files are used by the build tools to compile and process packages efficiently. This directory is managed by Colcon and the underlying build tools.
- install/ contains the installed software components, including executable files, libraries, Python modules, package resources, and environment setup scripts. These files allow the built packages to be accessed and used by ROS 2 applications. This directory is generated and managed by Colcon and the underlying build tools.
- log/ contains build logs, error messages, warnings, and diagnostic information generated during build and test operations. These records help developers identify the causes of build failures and troubleshoot problems within individual packages or the workspace as a whole. This directory is managed by Colcon.

Together, these four directories provide an organized workspace structure that separates the original source code from intermediate files, installed components, and diagnostic records, making ROS 2 development and package management easier to maintain.

V. CUSTOM INTERFACES

In ROS 2, custom interfaces are user-defined data structures that specify how information is represented and exchanged between nodes. They allow developers to define messages, services, and actions according to the requirements of a robotic application. These interfaces are language-independent at the definition level and can generate language-specific representations and type-support code for languages such as C and C++ and, where supported, Python.

Custom interfaces promote modularity and interoperability by providing a common data structure that different nodes can use, even when those nodes are implemented in different programming languages

A. Type Safety, Serialization, and Data Transmission

When data is exchanged between ROS 2 nodes, it must be converted into a format that can be transmitted and correctly interpreted by the receiving node. This involves serialization, communication through the middleware, and deserialization.

- Type Safety: Custom interfaces define the structure and data types of a message in advance. This helps ensure that nodes use a consistent format when

exchanging information and reduces errors caused by incompatible data types.

- Serialization and CDR: Serialization converts a message into a format suitable for transmission. ROS 2 commonly uses the Common Data Representation (CDR) format with DDS-based middleware. At the receiving end, deserialization converts the transmitted data back into a usable message.
- Type Compatibility: Publishers and subscribers must use compatible interface definitions to communicate correctly. ROS 2 and its middleware provide mechanisms to identify message types and check compatibility. If their definitions are incompatible, communication may fail, depending on the middleware and its configuration.

VI. Quality of Service

Quality of Service (QoS) in ROS 2 refers to a set of policies that determine how data is transmitted and received between nodes. Since ROS 2 uses DDS (Data Distribution Service) middleware for communication, QoS allows developers to control factors such as message reliability, data retention, and message delivery frequency.

A. Important Quality of Service Policies

Reliability determines whether the communication system prioritizes delivering messages reliably or allows some messages to be lost.

- Reliable: The middleware attempts to ensure that messages are delivered to the receiving node, using mechanisms such as retransmission when supported and applicable.
- Best Effort: Messages are transmitted without guaranteeing delivery, reducing the overhead associated with recovery. (as in the case of ROS1)

Durability determines whether a subscriber can receive previously published data when it begins listening to a topic.

- Volatile: Only messages published after the subscriber becomes connected are available through normal topic delivery.
- Transient Local: The publisher retains previously published messages for delivery to compatible late-joining subscribers, subject to its history and depth settings.

History and depth determine how many messages are stored by the middleware for a publisher or subscriber.

- Keep Last: Retains only the most recent specified number of messages.
- Keep All: Retains messages subject to the middleware's available resources and configured limits.

The depth specifies the number of messages retained when the Keep Last policy is used. For example, a depth of 10 allows the middleware to retain up to the last 10 messages within the applicable history queue.

These settings help balance memory usage against the need to retain messages when nodes process data at different rates.

Deadline specifies the expected maximum interval between consecutive messages on a topic. If the expected interval is missed, the middleware can report a deadline event.

Lifespan specifies how long a message remains valid for delivery. Messages that exceed the configured lifespan are considered expired and should not be delivered as valid new data. This is useful for time-sensitive information, such as robot position updates, where outdated data may no longer be useful for navigation.

B. 2. QoS Compatibility

For a publisher and subscriber to communicate successfully, their QoS settings must be compatible. If their policies are incompatible, the subscriber may not receive messages even when both nodes are running and using the same topic name.

For example, a reliable subscriber can connect to a reliable publisher or a best-effort publisher, but a best-effort subscriber cannot connect to a reliable publisher under the standard reliability compatibility rules.

Therefore, developers must select appropriate QoS settings for both publishers and subscribers to ensure that their communication requirements are satisfied.

C. 3. Importance of QoS in ROS 2

QoS allows developers to adapt communication behaviour to the requirements of different robotic applications. It helps balance message delivery, latency, memory usage, and data freshness. Selecting appropriate QoS policies is particularly important in systems involving sensors, autonomous navigation, robot control, and real-time monitoring.

Overall, QoS provides flexibility in ROS 2 communication by allowing nodes to exchange information according to the reliability and timing requirements of the application.

VII. Task One

Question 1 was to develop a ROS 2 publisher–subscriber system that calculates the longitudinal velocity of a wheel using its angular velocity and radius. The solution consists of two packages: taskone_interfaces (the custom message) and taskone_py, (the publisher and subscriber nodes).

Custom Message: A custom message named TaskOne.msg was created in the taskone_interfaces package. It contains two

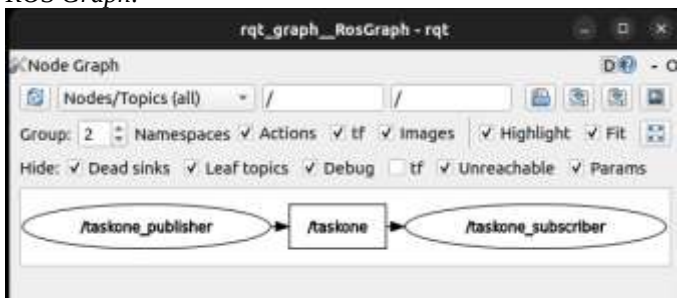
fields: ang_vel of type float64 to represent the wheel's angular velocity, and radius of type float64 to represent its radius. This message enables the publisher to transmit both values together over the /taskone topic.

Publisher Node: The publisher node prompts the user to enter the angular velocity and wheel radius through the terminal. It stores these values in a TaskOne message and publishes the message to the /taskone topic.

Subscriber Node: The subscriber node subscribes to /taskone and receives the published values. It calculates the longitudinal wheel velocity using the relationship: $v = \omega r$ where (v) is the longitudinal velocity, (ω) is the angular velocity, and (r) is the wheel radius. The calculated velocity is displayed in the terminal using the ROS 2 logger.

Execution and Result: Both nodes were executed successfully, demonstrating communication through the custom message. Whenever the publisher sends a new angular velocity and radius, the subscriber calculates and displays the corresponding longitudinal velocity.

ROS Graph:



VIII. TASK TWO

Question 2 was to develop a ROS 2 system that checks whether a user-entered string is a palindrome and returns a textual response indicating the result. The solution uses three nodes: a string publisher, a palindrome checker, and a result responder. The packages are rt2_interfaces, which defines the custom message, and rt2_py, which contains the three nodes.

Custom Message: A custom message named RT2String.msg was created in the rt2_interfaces package. It contains a single field, data of type string, which is used to transmit the input string and the final textual response.

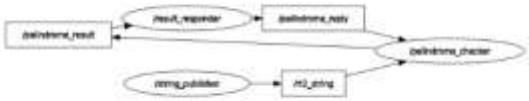
Node 1 String Publisher: The publisher continuously prompts the user to enter a string. It stores the input in an RT2String message and publishes it to the /rt2_string topic. The user can type exit to stop the publisher.

Node 2 Palindrome Checker: The palindrome checker subscribes to /rt2_string and checks whether the received string is equal to its reverse. This is implemented by comparing text with text[::-1]. If the strings match, the node publishes the integer 1 to /palindrome_result; otherwise, it publishes 0. The node uses the Int32 message type for this result. It also subscribes to /palindrome_reply to receive the final textual response.

Node 3 Result Responder: The result responder subscribes to /palindrome_result. If the received value is 1, it publishes the string "Yes"; if the value is 0, it publishes "No". The response is published to /palindrome_reply using the RT2String custom message.

Execution and Result: All three nodes were executed successfully and communicated through their respective topics. For example, when the input is madam, the palindrome checker publishes 1, and the result responder sends "Yes". For an input such as hello, the checker publishes 0, and the responder sends "No". The palindrome checker receives the final response and displays it in the terminal.

ROS Graph:



IX. CONCLUSION

In conclusion, ROS 2 provides a flexible and modular framework for developing robotic applications by enabling communication and coordination between different software components. Its core concepts, including nodes, topics, services, actions, and parameters, allow developers to structure applications efficiently and manage interactions between individual components. The computational graph represents these relationships, providing an understanding of how the various nodes communicate within a robotic system.

Additionally, custom interfaces enable the exchange of structured data between nodes, while Quality of Service (QoS) policies allow communication to be configured according to the requirements of different applications. Tools such as Colcon simplify the development process by managing package dependencies, building software, and organizing workspaces. Together, these features make ROS 2 suitable for developing scalable and maintainable robotic systems, including autonomous robots that require reliable communication, efficient data processing, and coordination between multiple components. Understanding these fundamental concepts provides a strong foundation for further exploration and implementation of ROS 2 in practical robotics applications.

Acknowledgment

I would like to express my sincere gratitude to my seniors at Formula Manipal for their guidance and encouragement throughout this task. Their insistence on technical rigor motivated me to delve deep into ROS2's underlying mechanics and complete this report.

References

[1] Open Robotics, "Nodes," *ROS 2 Documentation*. [Online]. Available: https://docs.ros.org/en/rolling/Concepts/Basic/About-Nodes.html

[2] Open Robotics, "Topics, Services, And Actions," *ROS 2 Documentation*. [Online]. Available: https://docs.ros.org/en/rolling/Concepts/Basic/About-Topics-Services-Actions.html

[3] Open Robotics, "About Parameters," Ros 2 Documentation. [Online]. Available: https://docs.ros.org/en/rolling/Concepts/Basic/About-Parameters.html

[4] Open Robotics, "Interfaces (Topics, Services, Actions)," Ros 2 Documentation. [Online]. Available: https://docs.ros.org/en/rolling/Concepts/Basic/About-Interfaces.html

[5] Open Robotics, "Colcon Tutorial," Ros 2 Documentation. [Online]. Available: https://docs.ros.org/en/rolling/Tutorials/Beginner-Client-Libraries/Colcon-Tutorial.html